

Model Abstraction Layer for Safer Agentic Access to Real Systems

Abstract and problem framing

Agentic systems (LLMs with tool access) are increasingly being connected to the places where “real work” happens—email, calendars, filesystems, SaaS applications, internal APIs, and developer tooling—because connected context and tools materially increase usefulness. ¹ Yet the same integration makes security failures higher-impact: protocols and connectors can expose arbitrary data access and “code execution paths”, and tool calls can become the mechanism by which unintended actions occur.

²

This white paper proposes a **Model Abstraction Layer (MAL)**: a deliberately *capability-attenuating* and *data-minimising* layer that sits between an LLM (or an LLM tool-connection protocol such as MCP) and fundamental resources (inboxes, calendars, storage, databases, line-of-business systems). The MAL is designed to look and feel like the underlying resource (a “transparent mode”), but it only exposes the minimum data and minimum actions required for a specific workflow—thereby reducing attack surface, limiting blast radius, and improving resilience in the face of prompt injection, model mistakes, and overly-broad permissions. ³

The MAL is intended to be composable with the **Model Context Protocol (MCP)**—for example: *MCP client* → *MAL (as an MCP server, or as a proxy)* → *underlying systems*—and can itself be layered (MAL-of-MAL) to progressively refine exposure. ⁴

Why agentic access is qualitatively riskier than “normal” integrations

Modern tool-using LLMs introduce failure modes that legacy systems were not designed to withstand—especially systems built for humans, coarse-grained app integrations, or trusted internal automation.

First, **prompt injection is a distinct social-engineering class targeting agents**: an attacker embeds malicious instructions in content the agent processes (webpages, documents, emails, tickets), aiming to override the user’s intent and induce the agent to leak data or misuse tools. ⁵ OpenAI explicitly characterises prompt injection as an “open challenge” for agent security and recommends limiting agent access to only the sensitive data or credentials required for the task (e.g., use logged-out modes when possible). ⁶ This is aligned with the OWASP emphasis on prompt injection (LLM01) and “Excessive Agency” (LLM06): damaging actions can be triggered by unexpected, ambiguous, or manipulated model outputs. ⁷

Second, **tool ecosystems create compositional risk**: even if individual tools appear safe, chaining them can enable high-impact attacks. In early 2026, multiple vulnerabilities in the official Git MCP server (`mcp-server-git`) demonstrated how argument injection and path validation issues could be exploited—potentially via prompt injection—and chained with filesystem access to enable file tampering or code execution. ⁸ This is not merely a theoretical concern; it is an empirical reminder

that “connector surfaces” are security-critical and that least-privilege boundaries must exist outside the model. ⁹

Third, **legacy authorisation is often too coarse for agent workflows**. Many systems are built around broad, user-level access (“read the inbox”, “access the drive”), because human users can (usually) be socially and procedurally constrained. Agentic workflows invert the risk: the system is exposed to a highly persistent, creatively problem-solving process that will keep acting until it believes the task is done, which increases the probability of unintended access and unintended side effects. This is consistent with OWASP’s framing of autonomous action risk and with MCP’s own explicit warnings about arbitrary data access and code execution paths. ¹⁰

The Model Abstraction Layer concept and core capabilities

The MAL is a *policy-defined façade* that sits “in front of” a resource. It can be inserted:

- **Directly in front of an asset** (e.g., email datastore, calendar API, filesystem, database), or
- **Between MCP layers**: an MPC client talks to an MAL-backed MCP server, which then talks to real systems (possibly via downstream MCP or conventional APIs). ⁴

Core principle: minimise both data exposure and capability exposure

The MAL’s central design goal is to shrink (1) **the data plane** and (2) **the capability plane** to the smallest surface consistent with the user’s requested workflow.

- **Data protection (read plane)**: expose only the minimum data needed for the task, consistent with data minimisation principles (collect/share “adequate, relevant and limited to what is necessary”). ¹¹
- **Capability protection (write/action plane)**: expose only the minimum actions needed for the task. This aligns with least privilege (“restrict access privileges... to the minimum necessary to accomplish assigned tasks”). ¹²

This dual-plane framing also maps cleanly to the two most common sources of agent harm: accidental or induced **sensitive information disclosure** and accidental or induced **damaging actions** via over-permissive tools. ¹³

The inbox example as a canonical “legacy system not fit for agentic exposure”

The transcript’s inbox example is a strong illustration of why MAL is needed: giving an agent access to an entire inbox is rarely what the user intends, and even “read-only” inbox access can be a stepping stone to broader compromise.

A compromised email account (or an agent misled into handling email content unsafely) can enable account takeovers via password resets: UK security guidance explicitly warns that if a hacker gets into your email, they can reset passwords for other accounts using “forgot password.” ¹⁴ Separately, OpenAI’s own prompt injection examples for agents repeatedly highlight the risk that a malicious email or webpage can trick an agent into retrieving password reset codes or other sensitive data and exfiltrating it. ¹⁵

Yet many user requests are narrow—e.g., “check my latest emails and give me an overview.” The MAL’s job is to offer an “inbox-like” interface while enforcing a policy such as: - Only emails from the last day/week/month (progressive trust).

- Filter out password reset emails (or any messages matching sensitive patterns).
- Remove or redact confidential fields before they reach the model. ¹⁶

Two operating modes: transparent and normalised

The MAL should support two complementary modes (both explicitly called out in the transcript):

Transparent mode (“looks just like the real thing”): the calling agent or tool layer interacts with a resource-shaped interface that resembles the original system, but the MAL enforces strict policy boundaries behind the scenes. This supports compatibility with existing tool schemas and MCP Resources/Tools patterns. ¹⁷

Normalised mode (“clean up the data a little bit”): instead of passing through raw artifacts (e.g., full MIME email payloads), the MAL emits a standard structured representation (e.g., a clean JSON schema with subject/sender/time/body excerpt and safety annotations). This aligns with guidance to use structured outputs to constrain data flow and reduce free-form channels for injection and unintended propagation. ¹⁸

Chaining and “abstraction layers of abstraction layers”

A practical benefit of MAL is that it supports composition: one MAL can sit behind another, each adding constraints or converting representations. This mirrors established “defence in depth” thinking (multiple controls to overcome, increasing attacker work factor and detection likelihood). ¹⁹ It also dovetails with Zero Trust goals of limiting lateral movement by enforcing per-request checks and minimising implicit trust. ²⁰

Reference architecture with MCP integration

Positioning MAL relative to MCP

MCP standardises how LLM applications connect to external context and tools, offering **Resources** (context/data), **Tools** (callable functions), and **Prompts** (templates/workflows), over a JSON-RPC based protocol. ²¹ MCP’s specification also explicitly calls out security principles: user consent and control, data privacy, tool safety, and sampling controls—while noting that the protocol itself cannot enforce these at the protocol layer, so implementers must build robust authorisation and consent mechanisms. ²²

The MAL can be viewed as a concrete mechanism to enforce these MCP principles, especially:

- **Data privacy:** the MAL prevents over-sharing by constraining what becomes an MCP Resource payload. ²³
- **Tool safety:** the MAL gates or attenuates tools so that “Tools represent arbitrary code execution” is operationally constrained to workflow-necessary operations. ²⁴
- **User consent and control:** the MAL can implement step-up consent (progressive trust) and human-in-the-loop approvals for high-impact operations. ²⁵

Minimal components of an MAL implementation

A robust MAL typically decomposes into five components:

Policy decision point and policy language: policies should be externalised (“policy as code”) so they can be reviewed, tested, versioned, and audited. ²⁶ This can leverage: - `entity["organization","Open Policy Agent","policy engine, cncf"]` (OPA) as a general-purpose policy engine with a structured policy language (Rego). ²⁶

- **Cedar** as an authorisation policy language and decision engine designed for expressive, safe, analysable policies, with tooling to analyse policies across scenarios. ²⁷
- ABAC concepts (subject/object/action/environment attributes) from NIST SP 800-162, to model fine-grained authorisation decisions. ²⁸

Adapters/connectors: per-resource connectors that translate from real systems (email providers, calendar APIs, file stores) to MAL’s internal representation and enforcement hooks, and optionally to MCP Resources/Tools. ¹⁷

Transformers/sanitizers: canonicalisation, redaction, classification, normalisation, and “safe summarisation” transforms, to reduce sensitive information disclosure risk and reduce injection payload surface. ²⁹

Capability gate and step-up authorisation: an explicit gate for write operations and other high-impact actions (send email, delete file, share document, modify calendar entry). This directly addresses Excessive Agency by preventing ambiguous model output from being interpreted as authority to act. ³⁰

Telemetry and audit: logs suitable for detection and incident response (what data was accessed, what was withheld, what actions were attempted/denied, what policies fired). This aligns with MCP’s mention of logging utilities and with Zero Trust’s emphasis on continuously evaluating and authorising each request. ³¹

Capability attenuation, not just allow/deny

Where integrations require delegating authority (for example, letting an agent run a limited workflow autonomously), the MAL should favour *attenuated delegation* rather than broad tokens. This is a well-known strength of “caveated” capability credentials such as **macaroons**, which can confine when/where/by whom/for what purpose a service should authorise requests, and can be tightened via additional caveats without weakening integrity. ³²

In practice, MAL can implement capability attenuation in multiple ways, depending on environment maturity:

- **Ephemeral, workflow-bounded capability tokens** (time-bounded, scope-bounded, object-bounded). ³³
- **“Two-man rule” / human confirmation** for destructive actions, consistent with MCP’s consent-and-control principle and common agent safety patterns. ³⁴
- **Read-only-by-default** with explicit step-up to write, mirroring least-privilege scope design in modern authorisation frameworks. ³⁵

Security model and policy design patterns

Data minimisation as a first-class engineering invariant

Data minimisation is not only a compliance principle; it is a practical security design tool. The UK GDPR framing (via the ICO) is direct: identify the minimum personal data needed for the purpose and hold/share no more. ³⁶ For agentic systems, the MAL turns “purpose limitation” into executable policy: each workflow declares its purpose, and policy constrains which records, fields, and time windows are in-scope. ³⁷

Concrete examples (explicitly preserving your transcript’s ideas):

- **Inbox overview workflow:** expose only the last N days of email; exclude password-reset messages and other high-risk categories; provide only normalised fields (subject, sender, timestamp, short excerpt). ³⁸
- **Calendar “what’s next” workflow:** expose only upcoming events for the next week; strip attendee emails unless required; redact meeting links unless actioned. ³⁹
- **Filesystem “summarise this folder” workflow:** expose a single directory tree, not the whole disk; provide metadata and selected files only; deny dotfiles, keys, credential stores by policy. This is consistent with the practical lesson from MCP server vulnerabilities: filesystem adjacency can turn connector flaws into high-impact compromise. ⁴⁰

Least privilege and “granularity repair” for legacy authorisation

Least privilege is a long-established security principle: grant only the accesses necessary for tasks. ¹² The MAL acts as a “granularity repair layer” for systems whose native authorisation model is too coarse. Instead of giving the agent a top-level grant (entire mailbox, entire drive), the MAL becomes the *enforcement point* that constrains data and actions to task-shaped slices. This is conceptually aligned with ABAC (“evaluate subject/object/action/environment attributes against policy”) and with large-scale authorisation systems that separate enforcement from business logic. ⁴¹

Defence against injection and exfiltration: reduce surface, constrain channels

Because prompt injection is expected to remain a persistent industry-wide challenge, the MAL should assume models can be tricked sometimes and engineers must design for containment. ⁴² Two patterns are particularly important:

Reduce the accessible data/capability surface: OpenAI’s agent safety guidance explicitly recommends reducing attack surface and limiting sensitive access; MCP likewise emphasises user consent and careful tool handling. ⁴³

Constrain interfaces with structure: OpenAI’s agent-builder guidance recommends structured outputs between nodes to eliminate free-form channels that attackers can exploit. ⁴⁴ In MAL terms, normalised mode is not just convenience—it is an injection-resistance strategy.

Where additional controls are warranted, MAL can integrate classic **data loss prevention (DLP)** style checks: DLP is explicitly framed as identifying and helping prevent unsafe or inappropriate sharing/transfer of sensitive data. ⁴⁵ A MAL that performs output-boundary DLP checks before a tool call (e.g., before sending email, sharing a link, uploading a file) materially reduces accidental or induced data leakage. ¹³

Implementation blueprint: multi-agent build plan, roles, and test-first delivery

This section turns the concept into an implementable programme, explicitly designed to be executed by a **series of bot agents**, using a mix of OpenAI Codex and “cloud code” tooling such as Claude Code.

Delivery shape: an open reference implementation + exemplar connectors + documentation site

A credible open release should include:

- A core MAL runtime (policy evaluation, transforms, audit, step-up gate). ⁴⁶
- At least three “interesting implementations” (as you requested), showing the same ideas applied to different resource types: 1) Inbox summariser façade, 2) Calendar assistant façade, 3) Filesystem-limited research/summarisation façade. ⁴⁷
- MCP integration demonstrating MAL as a server that exposes MAL-defined Resources/Tools, with explicit consent and tool gating. ²¹
- A test suite: unit tests for policies/transforms, integration tests for connectors, and adversarial tests for injection-like payloads (shift-left). ⁴⁸

Agent roles (including “librarian”) and responsibilities

Below is a practical role decomposition that matches your requested roles and is compatible with modern agentic coding tooling.

Architect / Orchestrator / Conductor agent

Owns the system design, module boundaries, and sequencing. Produces an architecture spec, threat model, and the “Definition of Done” for each milestone. Should explicitly bake in MCP security principles (consent/control, data privacy, tool safety) and Zero Trust assumptions (no implicit trust, per-request checks). ³¹

Librarian agent (connect-the-dots + mapping)

Maintains a living knowledge base of decisions: how requirements map to patterns (least privilege, ABAC, defence in depth), how MAL maps to MCP constructs (Resources/Tools), and how each connector’s policy “purpose” is declared. Also curates citations and terminology consistency for the public white paper and site. This mirrors NIST AI RMF practice of structured risk thinking across lifecycle and provides continuity across many specialised agents. ⁴⁹

Implementation agents (developers, specialised by subsystem)

Split by subsystems: - Policy engine integration (OPA/Rego or Cedar evaluator). ⁵⁰
- MCP server/proxy integration and schema definitions. ²¹
- Connector implementations (email/calendar/filesystem). ⁵¹
- Transformation/normalisation pipeline (redaction, structured output). ¹⁸
- Audit/logging and policy traceability. ³¹

QA / Security test agents

Build automated tests and adversarial harnesses: - Policy unit tests (positive/negative cases). ⁵²
- Connector integration tests with mocked resources. ⁵³

- Prompt-injection regression corpus and “misuse case” tests inspired by OWASP and MITRE ATLAS. ⁵⁴
- “Canary” tests for high-impact actions (ensure step-up required; ensure denial pathways). ⁵⁵

Tooling fit: Codex + Claude Code as parallel implementation engines

The requested mix is technically well-matched to the project shape:

- OpenAI’s Codex supports multi-agent and parallel workflows (Codex app; cloud tasks running in isolated environments/worktrees). ⁵⁶
- Codex documentation explicitly highlights security defaults such as network access off by default and sandboxed operation, alongside cautions about enabling internet access due to prompt injection and exfiltration risk. ⁵⁷
- Claude Code is positioned as an agentic coding tool across terminal/IDE/desktop, and Anthropic describes “agent teams” that can work in parallel—useful for splitting the MAL into independent read-heavy tasks (spec writing, connector scaffolds, docs, tests). ⁵⁸

A practical division of labour is: - Use Codex Cloud for longer-running implementation threads (connectors, MCP integration, policy engine modules). ⁵⁹

- Use Claude Code “agent teams” to accelerate documentation, examples, and test harness generation, plus refactors that require fast local feedback loops. ⁶⁰

A concrete, test-first milestone plan designed for bot agents

This is written as a sequence of artefact-producing milestones that agent teams can execute without ambiguity.

Milestone: Define the MAL contract and policy schema

Deliver: - A canonical “Resource Contract” describing: allowed fields, time windows, query parameters, and redaction rules.

- A “Capability Contract” describing: which write/actions exist, required approvals, and rate limits.
- A self-describing endpoint (or MCP metadata) where the MAL can describe itself and its policy boundaries so the user can validate “is this good enough for what you want to do?”. MCP already emphasises capability discovery and metadata, making this approach ecosystem-aligned. ⁶¹

Milestone: Build the core MAL runtime (policy + transforms + audit)

Deliver: - Policy evaluation integrated with OPA or Cedar. ⁵⁰

- Transformation pipeline supporting transparent and normalised modes, explicitly adopting structured output constraints. ¹⁸
- Audit logging and policy traces, consistent with Zero Trust’s emphasis on continuous authorisation and monitoring. ²⁰

Milestone: Implement three exemplar façades (email/calendar/filesystem)

Deliver: - Inbox summary façade implementing the “last week only + exclude password reset / sensitive patterns” policy. ⁶²

- Calendar façade implementing “next 7 days only” with normalised event schema. ³⁶
- Filesystem façade implementing strict roots/boundaries and denying sensitive directories by default; this is directly motivated by evidence that filesystem adjacency can amplify connector vulnerabilities. ⁶³

Milestone: MCP integration and consent gates

Deliver: - MAL as an MCP server exposing Resources/Tools, with per-tool consent requirements and

explicit user-visible descriptions of tool behaviour (treating annotations as untrusted unless server is trusted, per MCP guidance). ²¹

- If MAL is deployed as a proxy in front of another MCP server, ensure it can enforce policy regardless of downstream server behaviour, reflecting the reality that MCP cannot enforce principles solely at protocol level. ²²

Milestone: Security regression suite (prompt injection + excessive agency)

Deliver: - OWASP-derived misuse cases for prompt injection and excessive agency. ¹³

- MITRE ATLAS mapping for the project's threat model and tests; ATLAS is positioned as a living knowledge base of adversary tactics/techniques against AI-enabled systems. ⁶⁴

- Automated "shift-left" prompt injection mitigation testing approach (e.g., as advocated in SANS material). ⁶⁵

Where your "Issues-FS-Pipe-I" project fits

You asked to manage development using your **issues, FS, pipe, I** project. Even without assuming its internal mechanics, the MAL programme naturally fits an issue-driven, artefact-centric agent workflow:

- **Issues:** each milestone decomposed into small, verifiable tickets with explicit artefacts (policy schema, connector module, test suite, doc page).
- **FS (filesystem as source of truth):** agents commit artefacts (specs, policy code, fixtures) as versioned files; structured docs reduce ambiguity and support reproducibility. ⁶⁶
- **Pipe (CI/CD pipeline):** every PR triggers policy tests, connector integration tests, and adversarial suites; publish site on green builds. ⁶⁷
- **I (integration/index/intent layer):** whichever "I" stands for in your system, MAL benefits from an explicit integration layer that indexes policies, workflows, and connectors so agents can discover the correct abstraction for a given purpose (mirroring MCP's discovery of Resources/Tools). ¹⁷

Publishing as a community white paper and website

A public-facing release should have two complementary deliverables: (1) a PDF-like white paper (this document can be the initial draft) and (2) a navigable website that functions like a "reading page" and contribution hub.

Website information architecture

A strong structure, designed for community comprehension and contribution, is:

- **Principles:** why legacy resources are risky for agents; data minimisation + least privilege; defence in depth; progressive trust; transparent vs normalised modes. ⁶⁸
- **Architecture:** MAL positioning relative to MCP; components (policy engine, transformers, capability gate, audit). ⁶⁹
- **Threat model and mitigations:** prompt injection, tool chaining, excessive agency; include a short case study of the MCP Git server vulnerabilities as a real-world demonstration of connector-layer risk. ⁷⁰
- **Reference implementations:** inbox/calendar/filesystem demos; show transparent mode vs normalised JSON mode outputs and how policies change the accessible surface. ⁷¹
- **Contributing:** how to add a connector, how to write a policy, how to add tests, how policy analysis/verification works (especially if Cedar analysis is adopted). ⁷²

Static documentation generators (e.g., Docusaurus, MkDocs) are commonly used to publish developer-focused documentation sites with versioning and search, and would suit this “presentation-like reading flow.” ⁷³

Community credibility: emphasise testing, not just philosophy

Because prompt injection and agentic misuse are ongoing challenges, the community will look for evidence of *operational* safety. ⁷⁴ Publishing a strong test suite is therefore central to the project’s legitimacy: - Include a “known-bad” corpus and demonstrate how MAL policies constrain blast radius even when the model is fooled. ⁷⁵

- Publish a threat-model mapping to MITRE ATLAS techniques and keep it updated as ATLAS evolves. ⁶⁴

- Treat security as compositional: add tests that combine connector capabilities (because real failures often arise from composition). ⁷⁶

Positioning statement for the community

A crisp message, faithful to the transcript and supported by current security guidance, is:

- MCP standardises connection—but implementers still need strong consent, least privilege, and safe tool boundaries. ⁷⁷
- Prompt injection and excessive agency make “broad access” unsafe; we must engineer for containment by default. ⁷⁸
- MAL provides that containment by making *workflow-specific, policy-defined abstractions* the default way models touch real systems—reducing exposed data, reducing enabled capability, and enabling progressive trust. ⁷⁹

This is, in effect, a practical bridge between established security ideas (least privilege, ABAC, defence in depth, zero trust) and the unique realities of agentic automation. ⁸⁰

1 Introducing the Model Context Protocol \ Anthropic

<https://www.anthropic.com/news/model-context-protocol>

2 4 21 22 23 24 25 31 34 69 77 Specification - Model Context Protocol

<https://modelcontextprotocol.io/specification/2025-11-25>

3 7 13 29 54 OWASP Top 10 for Large Language Model Applications

https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com

5 6 16 42 74 Understanding prompt injections: a frontier security challenge

https://openai.com/index/prompt-injections/?utm_source=chatgpt.com

8 Missing path validation when using --repository flag in mcp ...

https://github.com/modelcontextprotocol/servers/security/advisories/GHSA-j22h-9j4x-23w5?utm_source=chatgpt.com

9 40 63 70 76 Anthropic MCP Server Flaws Lead to Code Execution ...

https://www.securityweek.com/anthropic-mcp-server-flaws-lead-to-code-execution-data-exposure/?utm_source=chatgpt.com

10 30 55 78 LLM06:2025 Excessive Agency - OWASP GenAI Security Project

https://genai.owasp.org/llmrisk/llm06-sensitive-information-disclosure/?utm_source=chatgpt.com

- 11 36 37 39 68 79 **Principle (c): Data minimisation | ICO**
https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-protection-principles/a-guide-to-the-data-protection-principles/data-minimisation/?utm_source=chatgpt.com
- 12 80 **least privilege - Glossary - NIST CSRC**
https://csrc.nist.gov/glossary/term/least_privilege?utm_source=chatgpt.com
- 14 38 47 62 **Cyber aware National Cyber Security Centre alternative ...**
https://www.lead.org.uk/wp-content/uploads/2021/05/FINAL-Cyber-Aware-Alternative-Formats-Masterv0.4.pdf?utm_source=chatgpt.com
- 15 75 **Continuously hardening ChatGPT Atlas against prompt ...**
https://openai.com/index/hardening-atlas-against-prompt-injection/?utm_source=chatgpt.com
- 17 51 53 61 **Resources - Model Context Protocol**
<https://modelcontextprotocol.io/specification/2025-06-18/server/resources>
- 18 43 44 66 71 **Safety in building agents | OpenAI API**
https://platform.openai.com/docs/guides/agent-builder-safety?utm_source=chatgpt.com
- 19 **PL-8(1): Defense in Depth**
https://csf.tools/reference/nist-sp-800-53/r5/pl/pl-8/pl-8-1/?utm_source=chatgpt.com
- 20 **Zero Trust Architecture - NIST Technical Series Publications**
https://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.SP.800-207.pdf?utm_source=chatgpt.com
- 26 46 50 **Open Policy Agent (OPA)**
https://openpolicyagent.org/docs?utm_source=chatgpt.com
- 27 **What is Cedar? | Cedar Policy Language Reference Guide**
https://docs.cedarpolicy.com/?utm_source=chatgpt.com
- 28 41 **SP 800-162, Guide to Attribute Based Access Control (ABAC ...**
https://csrc.nist.gov/pubs/sp/800/162/upd2/final?utm_source=chatgpt.com
- 32 33 **Macaroons: Cookies with Contextual Caveats for ...**
https://theory.stanford.edu/~ataly/Papers/macaroons.pdf?utm_source=chatgpt.com
- 35 **AC-6: Least Privilege**
https://csf.tools/reference/nist-sp-800-53/r5/ac/ac-6/?utm_source=chatgpt.com
- 45 **What is data loss prevention (DLP)?**
https://www.microsoft.com/en-gb/security/business/security-101/what-is-data-loss-prevention-dlp?utm_source=chatgpt.com
- 48 65 67 **Reducing Prompt Injection Risk with 'Shift-Left' MITRE ...**
https://www.sans.org/white-papers/automating-generative-ai-guidelines-reducing-prompt-injection-risk-shift-left-mitre-atlas-mitigation-testing?utm_source=chatgpt.com
- 49 **Artificial Intelligence Risk Management Framework (AI RMF 1.0)**
https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf?utm_source=chatgpt.com
- 52 72 **Introducing Cedar Analysis: Open Source Tools for ...**
https://aws.amazon.com/blogs/opensource/introducing-cedar-analysis-open-source-tools-for-verifying-authorization-policies/?utm_source=chatgpt.com
- 56 **Introducing the Codex app**
https://openai.com/index/introducing-the-codex-app/?utm_source=chatgpt.com
- 57 **Security**
https://developers.openai.com/codex/security/?utm_source=chatgpt.com

58 **Claude Code overview - Claude Code Docs**

https://code.claude.com/docs/en/overview?utm_source=chatgpt.com

59 **Codex web**

https://developers.openai.com/codex/cloud/?utm_source=chatgpt.com

60 **Introducing Claude Opus 4.6**

https://www.anthropic.com/news/claude-opus-4-6?utm_source=chatgpt.com

64 **MITRE ATLAS™**

https://atlas.mitre.org/?utm_source=chatgpt.com

73 **Language Server Protocol - Microsoft Open Source**

https://microsoft.github.io/language-server-protocol/?utm_source=chatgpt.com